

9.a.File Handling

Stream

What is a stream:- A stream is flow of data.

```
1 cout<<"Hello, World!"<<endl
```

 **Flow**

Data ----> Screen

```
1 std::cin >> x;
```

 **Flow**

Keyboard ----> Program

Arrow confusion

- only for cramming purposes

scheme	flow
lhs >> rhs	lhs → rhs
lhs << rhs	rhs → lhs

flow of the data is towards the pointing side

```
1 cat          hello.txt > newFile.txt
2 read-file
3 echo  "Hello, World!" > hello.txt
4 printf(      )
```

now u suddenly understand bash too

streams(the data containers) are always present towards the **left** side. so the above commands now make sense. (Why **echo** && **cat** are present on the left side)

```

1  cout<<"Hello, World!"<<endl;
2  cin >> n;

```

all (data sources) present on the **left** (`cout` && `cin`) side.

Concluding

scheme	what is actually happening
>>	stream gives the data
<<	stream receives the data

The threes

- same idea different methods

Class	Purpose
<code>ifstream</code>	Read from a file
<code>ofstream</code>	Write to a file
<code>fstream</code>	Read and write

THESE ALL THERE THINGS COME FROM THE `fstream` HEADER

ifstream

```

1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      string str;
7      ifstream file("data.txt");
8          /* two ways to interpret the line 7
9          1. Creating the file object
10         2. Creating a stream of data originated from the file
11         */
12
13         //use any one of the below
14         if(!file){
15             cout<<"NOT able to open the file"<<endl;
16             return 1;
17         }
18         //OR

```

```

19     if(!file.is_open()){
20         cout<<"Not able to open the file"<<endl;
21         return 1;
22     }
23
24     //stream on the left
25     file >> str;
26
27     file.close(); //good idea to clear your mess
28
29     /*
30         flush buffers
31         release file handle
32         disconnect stream from file
33     */
34     return 0;
35 }

```

- this will read till there is a **white space** character (`\t` `\n` `' '`)

ofstream

- this will over-write the whole file.

```

1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      string str;
7      ofstream file("data.txt");
8
9      if(!file.is_open()){
10         cout<<"Not able to open the file"<<endl;
11     }
12
13     //stream on the left
14     file << "Medhansh is a good boy.";
15     return 0;
16 }

```

fstream

```

1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4

```

```

5  int main(){
6      string str;
7
8      fstream file("data.txt",ios::in | ios::out);
9      //same as fstream file("data.txt");
10
11
12     //stream on the left
13     file >> str;
14     file.close();
15
16     return 0;
17 }

```

Mode	Meaning	If file initially contains <code>Hello</code> and we write <code>"ABC"</code>
<code>ios::in</code>	Read only	Can't write
<code>ios::out</code>	Write (overwrite)	<code>ABC</code>
<code>ios::app</code>	Append to end	<code>HelloABC</code>
<code>ios::ate</code>	Start at end	1. open the file and immediately place the current position at the end 2. then move the pointer as you wish using <code>seekp()</code>
<code>ios::trunc</code>	Erase file on open	<code>ABC</code>
<code>ios::binary</code>	Raw bytes mode	Content depends on what bytes are written

```

1  std::fstream file("data.txt", std::ios::in | std::ios::out |
2  std::ios::ate);
3  seekp(0);
4  file << "ABC";

```

 **Final file**

ABClo

in **app** you can't actually move the write pointer.**

Seek and Tell

Function	Used for	Meaning
<code>seekg()</code>	Input (reading)	Move the read pointer
<code>tellg()</code>	Input (reading)	Get current read position
<code>seekp()</code>	Output (writing)	Move the write pointer
<code>tellp()</code>	Output (writing)	Get current write position

The suffixes mean:

g = get (read)

p = put (write)

tell :- tell me the position

seek :- go to a position

seekg -> move/set input (get) pointer

seekp -> move/set output (put) pointer

tellg -> get current position of input (get) pointer

tellp -> get current position of output (put) pointer

Stream States

- status of the stream
- don't want to read just read the `eof()`

`good()`

```

1  if (file.good())
2  {
3      std::cout << "All good\n";
4  }
```

`eof()`

- returns true if you go past the end

```

1  file.eof();
```

`fail()`

```

1  int x;
2  /*
3  some file content
4  */
```

```

5
6 file >> x; // file had some string rather than the int
7
8 file.fail() //→ true
9

```

bad()

- Serious low-level I/O error.
1. disk failure
 2. hardware problem
 3. file corruption
 4. OS-level error

Bit advanced section (why better)

```

1 while (!file.eof())
2 {
3     ...
4 }

```

```

1 while (file >> x)
2 {
3     std::cout << x << '\n';
4 }
5 file.clear();

```

- **Second** one is better why :- `file>>x` returns the stream, which will automatically handle the eof, invalid input, failures.
- Why we need to use **clear** :- Once you use the above code will reach the EOF and further reading will lead to errors so we have to use clear.
 - what does this clear clear?
 - all the flags we discussed in the section `fail` , `eof` , `bad`

State	Meaning
<code>good()</code>	Everything okay
<code>eof()</code>	Tried reading past end
<code>fail()</code>	Read/write operation failed
<code>bad()</code>	Serious system-level error

Some Important functions you need to know

getline()

- exists in the namespace std

```
1  #include <iostream>
2  #include <fstream>
3
4  using namespace std;
5  int main(){
6      string str;
7      getline(file, str); //reads the whole line
8  }
```

```
1
2  //read line by line until the end
3  while (std::getline(file, line))
4  {
5      std::cout << line << '\n';
6  }
```

stringstream

- exists in the namespace std too;
- make a line to a stream
- buy why?
 - The reason being:- to use `>>` operator
 - But why to use
 - so we can extract word by word

```
1  stringstream ss (line);
2  line>>x; // read word by word
```